

# 7장 밤에 한 번 더 세는 이유 — 배치와 확정 통계

## 6주차 · 배치 처리

6장에서 스트리밍(streaming)은 지금 벌어지는 일을 빠르게 세는 대신, 워터마크(watermark)를 넘겨 늦게 도착한 이벤트를 폐기했다. 이 장은 그 다음 날 새벽에 같은 데이터를 다시 세는 일을 다룬다. 배치(batch)는 느린 대신 마감까지 도착한 것을 전부 담고, 한 번 낸 수치를 바꾸지 않는다.

이 장의 수치는 민원 이벤트 사흘치(물리 레코드 60건)를 실제 Airflow 3.3.0 엔진으로 돌린 결과다.

## 1. 오늘의 질문

- 6장에서 이미 실시간으로 세고 있는데, 밤에 왜 한 번 더 세는가?
- 원천 60건이 보고서에서 54건이 되었다면, 나머지 6건은 어디로 갔다고 답할 수 있는가?
- 배치가 중간에 실패했는데 보고서 파일이 만들어져 있다면, 무엇이 잘못된 것인가?
- 같은 날짜를 다시 계산했더니 수치가 달라졌다면, 어느 수치가 진짜인가?

## 2. 장부 마감으로 보는 이 장

가게가 하루를 마감하는 절차와 이 장의 배치는 항목이 하나씩 대응한다.

| 장부 마감                                   | 이 장의 배치                           |
|-----------------------------------------|-----------------------------------|
| 영업 중 계산대 화면에 뜨는 잠정 매출                   | 스트리밍 상황판의 근사치(6장)                 |
| 문을 닫은 뒤 그날 전표를 전부 다시 세어 만든 일계표          | 마감 후 전량을 재계산한 확정 통계               |
| 전표는 한 장도 버리지 않고 정상·기재누락·코드없음·중복으로 나눠 둔다 | 정제의 네 갈래 출구(4절)                   |
| 세어 본 합이 전표 뭉치와 안 맞으면 마감하지 않는다           | 총계 보존식 위반 시 배치를 실패시키는 품질 게이트(5절)  |
| 일계표를 모아 월계표를 만들 때 전표를 다시 세지 않는다         | 주간 롤업(rollup)은 일별 확정 산출물만 읽는다(6절) |

**비유가 깨지는 지점:** 장부는 사람이 마감하므로 합이 안 맞으면 조정 전표를 끼워 넣어 억지로 맞출 수 있다. 배치는 그렇게 하지 않는다. 사전에 없는 지역 표기를 "가장 비슷한 자치구"로 바꿔 합을 맞추는 것은 데이터 조작이며, 이 장의 설계는 맞추는 대신 **멈춘다**.

### 3. 확정치와 근사치는 다른 산출물이다

같은 민원을 세는데 왜 두 시스템이 필요한지는 산출물의 성격에서 갈린다. 상황판의 수치는 빨라야 하지만 나중에 바뀌어도 된다. 반대로 기관이 제출한 통계는 늦어도 되지만 제출한 뒤에 바뀌면 안 된다. 하나의 시스템으로 두 요구를 동시에 만족시킬 수 없기 때문에 스트리밍과 배치를 나란히 둔다.

마감을 정하는 주체가 다른 것이 두 방식의 핵심 차이다. 스트리밍에서는 **데이터가 마감을 정한다**. 6장의 워터마크는 지금까지 관측된 가장 늦은 이벤트 시각에서 허용 지연만큼을 뺀 시점을 마감으로 삼았고, 그 선을 넘겨 도착한 21:49 이벤트는 폐기되어 집계에 영원히 들어가지 못했다. 배치에서는 **달력이 마감을 정한다**. "7월 1일 데이터는 7월 2일 새벽에 마감한다"고 정해 두면, 그 시각까지 도착한 지연분은 늦었든 아니든 전부 포함된다.

여기서 배치의 세 가지 성질이 따라 나온다. 첫째는 **확정성**이다. 구간 전체를 한 번에 계산하므로 산출물이 "그 입력에 대한 최종 결과"가 되고, 스트리밍이 실시간으로 낸 근사치를 사후에 검증하는 기준선 역할을 한다. 둘째는 **전체 조망**이다. 그날 데이터를 통째로 손에 쥐고 있으므로 전체 정렬, 중복 전수 검사, 원천과의 대사가 자연스럽게 가능하다. 스트리밍은 흐르는 데이터의 창(window)만 보므로 이런 검사를 하기 어렵다.

셋째는 **재실행 가능성**이다. 배치는 실패하면 그 날짜를 통째로 다시 돌린다. 다만 다시 돌린 결과가 기존 산출물 위에 이중으로 쌓이면 안 되므로, 5장에서 다룬 멍등성이 배치의 전제 조건으로 되 돌아온다. 이 셋의 대가가 지연이다. 일 배치라면 산출물은 최대 하루 늦고, 그 사이의 상황 판단은 여전히 스트리밍이 맡는다.

감사 관점에서 중요한 것은 두 수치가 다를 수 있다는 사실 자체가 아니다. **왜 다른지가 문서로 설명되면 결함이 아니라 설계고, 설명되지 않으면 결함이다**.

### 4. 총계 보존식 — 모든 레코드의 행방을 적는다

정제에서 가장 위험한 코드는 처리하지 못한 레코드를 조용히 건너뛰는 코드다. 4장의 유실도, 6장의 워터마크 폐기도 스스로를 알리지 않았다. 정제 단계의 누락도 마찬가지여서, 보고서 총계가 원천보다 조금 작은데 아무도 모르는 상태가 된다. 감사에서 가장 곤란한 결함이 이것이다.

문제가 생기는 자리는 지역명 표기다. 7월 1일 파일에는 강남구 민원이 "강남구", " 강남구 " (앞뒤 공백), "강남" (약칭) 세 가지로 적혀 있다. 사람 눈에는 같은 곳이지만 이름을 키로 집계하면 강남구가 세 줄로 쪼개진다. 그래서 이름 대신 행정표준코드(법정동코드)를 키로 쓴다. 강남구는 11680, 마포구는 11440, 관악구는 11620이며, 코드를 쓰면 인구·예산 같은 다른 기관 데이터와도 같은 키로 붙일 수 있다.

정제는 모든 입력 레코드를 네 갈래 중 하나로 보내고 어디로 갔는지 기록한다.

1. **매핑 성공(mapped)** — 표준 코드가 붙어 집계에 들어간다.
2. **지역 미기재(missing)** — 지역 필드가 비어 있다. 건수와 민원 ID를 따로 기록한다.
3. **매핑 실패(unmapped)** — 사전에 없는 표기다. 원문 그대로 따로 기록한다.
4. **중복 제거(dup\_removed)** — 같은 민원 ID의 재전송은 먼저 온 것만 남긴다(5장의 원칙).

네 갈래의 합이 입력과 맞는지 검사하는 식이 **총계 보존식**이다.

```
physical = mapped + missing + unmapped + dup_removed
valid    = physical - dup_removed = mapped + missing + unmapped
```

`physical` 은 파일에 들어온 물리 레코드 수, `valid` 는 중복을 뺀 유효 레코드 수다. `gate` 는 이 검사의 판정이고, `preservation_ok` 는 등식 성립 여부다. 사흘치 실측은 다음과 같다.

**표 7.1** 일별 정제 회계( `ch7_batch_run_report.json` 의 `quality` 블록)

| 날짜         | physical | dup_removed | valid | mapped | missing | unmapped | gate | preservation_ok |
|------------|----------|-------------|-------|--------|---------|----------|------|-----------------|
| 2026-07-01 | 20       | 0           | 20    | 18     | 1       | 1(판교)    | PASS | true            |
| 2026-07-02 | 18       | 1           | 17    | 16     | 0       | 1(성남시)   | PASS | true            |
| 2026-07-03 | 22       | 0           | 22    | 20     | 1       | 1(김포)    | PASS | true            |

7월 1일을 손으로 검산하면  $20 = 18 + 1 + 1 + 0$  이고, 매핑된 18건은 강남구 8·마포구 5·관악구 5로 다시 나뉘어  $8+5+5=18$ 이 된다. 7월 2일은 같은 민원 ID의 재전송 1건이 중복으로 제거되어 물리 18건이 유효 17건이 되었고,  $18 = 16 + 0 + 1 + 1$ 로 등식이 유지된다. 사흘 전체로는 원천 60건이 매핑 54, 미기재 2, 미매핑 3, 중복 제거 1로 남김없이 설명된다.

**미기재와 미매핑을 합쳐 "실패 2건"으로 보고하면 안 된다.** 둘은 원인도 조치도 다르기 때문이다. 미기재는 접수 화면에서 지역을 받지 못했다는 뜻이므로 조치 대상은 접수 시스템이다. 미매핑은 값이 들어왔는데 우리 사전에 없다는 뜻이므로 조치 대상은 사전이다. 실제로 걸린 판교·성남시·김포는 오타가 아니라 서울 자치구 사전 밖의 유효한 지명일 수 있고, 관할이 넓어졌는지 접수 오류인지는 사람이 판단해야 한다. 두 항목을 합치면 그 판단에 필요한 정보가 사라진다.

각 항목은 그대로 운영 지표가 된다. 매핑률( $\text{mapped} \div \text{physical}$ )이 떨어지면 사전에 없는 표기가 늘었다는 신호이며, 관할이 실제로 넓어졌거나 접수 화면이 바뀌어 새 표기가 들어오기 시작한 것이다. 다만 비율만으로는 원인을 알 수 없으므로 그날의 미매핑 원문 목록(`excluded.json`)을 함께 본다.

## 5. DAG와 품질 게이트 — 틀린 보고서를 아예 만들지 않는다

정제와 집계와 보고서 생성을 한 스크립트에 이어 붙이면 운영에서 곧 막힌다. 어제 새벽 배치가 집계 도중에 죽었는데 어디서부터 다시 돌려야 하는지, 정제가 실패했는데 보고서 단계가 어제 파일을 읽고 그냥 돌아 버리지는 않았는지 알 수 없다. 워크플로 오케스트레이터(orchestrator)는 이 문제를 절차의 구조를 바꿔 푼다.

Airflow에서는 절차를 **\*\*태스크(task)\*\***와 **의존성**의 그래프로 선언한다. 이 그래프를 DAG(Directed Acyclic Graph, 방향 비순환 그래프)라고 부른다. 방향은 "정제가 끝나야 집계한다"는 순서를 뜻하고, 비순환은 "A가 B를 기다리고 B가 A를 기다리는" 교착이 없다는 보장이다. 순서 보장·실패 전파·재시도·스케줄은 엔진이 맡는다. 6장에서 질의만 선언하면 증분 계산을 엔진이 맡았던 것과 같은 구도다.

```
graph LR
    validate_input --> clean_and_map
    clean_and_map --> aggregate_daily
    clean_and_map --> write_report
    clean_and_map --> check_quality
```

```
t_validate >> t_clean >> [t_aggregate, t_quality] >> t_report
```

집계와 품질 점검은 둘 다 정제 결과만 있으면 되고 서로를 필요로 하지 않으므로 의존성을 선언하지 않았다. 그래서 이 둘의 실행 선후는 실행마다 달라지며, 이것은 오류가 아니다. **선언하지 않은 순서는 엔진이 보장하지 않는다**. 반대로 보고서는 둘 다 필요하므로 두 갈래가 다시 합쳐진다.

태스크 경계를 어디에 둘지는 두 가지가 정한다. 하나는 **재실행 단위**다. 태스크는 실패했을 때 다시 돌리는 최소 단위이므로, 10분 걸리는 정제와 집계를 한 태스크로 묶으면 집계에서 죽을 때마다 정제 10분을 다시 낸다. 다른 하나는 **실패 격리**다. 입력 검증은 파일이 없어서 실패하고 품질 점검은 데이터가 규칙을 어겨서 실패하는데, 둘을 묶으면 실패 상태만 보고는 원인을 구분할 수 없다. `check_quality`를 집계에서 떼어 낸 이유가 이것이다.

`check_quality`가 하는 일은 4절의 총계 보존식 검사다. 등식이 깨지면 이 태스크는 스스로 예외를 던지고, 하류의 `write_report`는 실행되지 않는다. **틀린 보고서를 만든 뒤 회수하는 것이 아니라 애초에 만들지 않는다는 것이 품질 게이트의 목적**이다.

실패가 어떻게 전파되는지는 입력 파일이 없는 7월 4일 실행에서 그대로 보인다. `validate_input`이 예외로 `failed`가 되고, 나머지 네 태스크는 모두 `upstream_failed` 상태로 아예 실행되지 않으며, DAG 실행 전체가 `failed`로 기록된다. 읽을 것이 둘이다. 하류가 실행되지 않았으므로 절반쯤 쓰다 만 보고서 파일이 남지 않고, 실패가 상태로 남으므로 4장의 조용한 유실이나 6장의 조용한 폐기와 달리 이 실패는 시끄럽다. 상태 기록이 없으면 "돌지 않았다"와 "돌 필요가 없었다"를 구분할 수 없다.

마지막으로 **logical date**는 실행이 처리하는 데이터 구간의 날짜다. 실습에서 7월 1일 데이터를 처리한 실행의 logical date는 2026-07-01이지만 실제로 돌아간 날은 2026-07-07이다. 2장의 이벤트 시간과 처리 시간 구분이 배치로 오면 "데이터의 날짜"와 "실행된 시각"의 구분이 된다. 이 이름표가 있어야 "7월 1일 실행을 다시 돌려라"가 어느 입력을 읽고 어느 산출물을 덮어쓸지 확정된 명령이 된다.

## 6. 재현성과 롤업 — 다시 계산해도 같은 바이트

배치를 다시 돌릴 수 있다는 말은 다시 돌린 결과가 원래 것과 같을 때만 의미가 있다. 감사에서 "그 통계를 다시 만들어 보라"는 요구를 받았는데 값이 달라지면, 어느 쪽이 진본인지를 두고 다투게 된다. 그래서 실습은 7월 1일 산출물의 해시(hash)를 기록해 두고 같은 날짜를 다시 돌린 뒤 비교한다.

daily\_summary.json 은 88a34af5412b4723..., report.md 는 ba2787da6c5b8a47... 로 재실행 전후가 바이트 단위까지 같았다.

이 결과는 저절로 나오지 않고 세 가지 설계에서 나온다. 첫째, 보고서에 생성 시각 같은 비결정 요소를 넣지 않아 내용이 입력만의 함수가 되게 한다. 둘째, JSON 직렬화의 키 순서를 고정해 같은 내용이 다른 바이트가 되는 일을 막는다. 셋째, 산출물을 이어 붙이지 않고 덮어쓴다. 덮어쓰기는 5장 Upsert의 파일판이며, 재실행이 이중 집계를 만들지 않게 하는 장치다.

재현성은 한 번 확인하고 끝나는 성질이 아니다. 정제 규칙이나 보고서 서식을 고치면 그날부터 같은 입력이 다른 바이트를 낸다. 그 변경이 의도한 것이라면 "확정치의 계산 규칙이 바뀐 날짜"로 기록되어야 하고, 의도하지 않은 것이라면 사고다. 해시 비교는 둘을 가르는 장치다.

주간 롤업은 여기서 한 층 위로 올라간다. **각 계층은 아래 계층의 확정 산출물만 읽는다.** 주간 집계가 원천 파일을 다시 정제하면 비용을 두 번 낼 뿐 아니라, 정제 규칙 버전이 어긋나는 순간 일별 합과 주간 값이 달라진다. 그러면 두 보고서가 모두 "제대로 계산되었는데" 서로 안 맞는 상황이 된다.

어느 날짜가 확정되었는지는 \_SUCCESS 마커(marker)로 판별한다. 일별 DAG의 첫 태스크가 실행을 시작할 때 마커를 지우고 마지막 태스크가 성공했을 때만 다시 쓰므로, 마커가 있다는 것은 그날의 배치가 품질 게이트까지 통과해 끝났다는 뜻이다. 주간 롤업은 마커가 있는 날짜만 합산하고 없는 날짜는 결측으로 기록한다. 실습의 주간 창(2026-06-29~07-05)에서 마커가 있던 날은 사흘뿐이었고 나머지 나흘은 결측으로 남았다.

사흘 합산 결과는 물리 60건에 매핑 54, 미기재 2, 미매핑 3, 중복 제거 1로 총계 보존이 성립했다. 지역별로는 강남구 23·마포구 16·관악구 15이고, 유형별로는 도로 21·소음 15·환경 10·청소 8이다. 지역별 합(23+16+15)과 유형별 합(21+15+10+8)이 모두 54로 매핑 건수와 맞는지 확인하는 것이 롤업 단계의 검산이다.

## 7. 실습 7.1 일별 민원 요약 DAG 작성 (난이도 ★★★★★)

### 실습 목표

- 5태스크 일별 DAG와 주간 롤업 DAG를 작성해 실행하고, 의존성 순서·실패 전파·재실행 재현성을 각각 증거로 확인한다.
- 실행하기 전에 입력이 없는 날짜의 태스크 상태를 예측하고, 실행 뒤 실측과 대조한다.

### 실행 환경

Python 3.10 이상(실습 검증 3.13.7)과 macOS 또는 Linux가 필요하다. Airflow가 Windows 네이티브를 지원하지 않으므로 Windows에서는 WSL2나 컨테이너(container)를 쓴다. 의존성이 많으므로 반드시 공식 constraints와 함께 설치한다.

## 시작 전 준비

Airflow는 **macOS·Linux 또는 Windows의 WSL2**에서만 실행된다. Windows 터미널에서 그대로 실행하면 설치 단계에서 실패한다. 아래 한 줄이 실행 환경과 네트워크를 확인하고 준비한다.

```
python scripts/setup_practice.py 7
```

설치할 때 Airflow 제약 파일을 내려받으므로 네트워크 연결이 필요하다.

## 실행 명령

```
cd practice/chapter7
python3.13 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt \
    --constraint https://raw.githubusercontent.com/apache/airflow/constraints-3.3.0/constraints-3.13.txt
python run_chapter7.py
```

## 핵심 코드

```
name = (raw or "").strip()           # 공백 보정
name = ALIASES.get(name, name)        # 약칭 보정: "강남" → "강남구"
if name in mapping: mapped.append(...) # 표준 코드 부여
elif not name:      missing.append(cid) # 미기재 - 분리 기록
else:               unmapped.append(...) # 매핑 실패 - 원문 보존
```

전체 코드는 *practice/chapter7/code/7-1-daily-report-dag.py* 참고

## 실행 전 예측

다섯 태스크( `validate_input`, `clean_and_map`, `aggregate_daily`, `check_quality`, `write_report` )가 입력 파일이 없는 7월 4일에 각각 어떻게 될지 적는다. 상태 이름을 모르면 "실행됨 / 실행 안 됨"으로 적어도 된다. 이어서 한 문장 더 적는다 — **이 실행이 실패하면 다시 돌려야 하는 범위는 어디까지인가.**

## 실행 결과 확인 사항

네 시나리오가 차례로 돌아간다. A는 7월 1~3일 일별 실행, B는 7월 1일 재실행 후 해시 비교, C는 입력이 없는 7월 4일 실행, D는 주간 롤업이다.

- A: 사흘 모두 `dagrun=success` 이고, 일별 회계가 표 7.1과 같다.
- B: 재실행 후 두 산출물의 해시가 동일하다(산출물 동일=True).
- C: `validate_input` 만 `failed` 이고 나머지 넷이 `upstream_failed` 다.
- D: 주간 합산이  $60 = 54 + 2 + 3 + 1$ 로 보존되고, 지역별이 23·16·15다.
- 마지막 줄이 `CH7_RUN_PASS` 다.

## 실측과 대조

예측을 "다섯 개 모두 failed"로 적었다면 하류 넷은 실행됐다가 실패한 것이 아니라 아예 실행되지 않았다는 점을 놓친 것이다. 갈리는 지점은 산출물이다. 실행됐다면 절반쯤 쓰다 만 보고서가 남아 다음 롤업에 섞여 들어간다.

재실행 범위의 답은 "그 날짜의 실행 전체"다. 다른 날짜는 자기 logical date의 입력만 읽었으므로 무관하다. 다만 7월 4일이 주간 창 안에 들어가면 롤업도 다시 돌려야 한다. `_SUCCESS` 가 생기는 순간 합산 대상이 하루 늘어 주간 수치가 바뀌기 때문이다.

한 가지 더 확인할 것은 입력 파일 없음이 **재시도로 낫지 않는 실패**라는 사실이다. 원천이 몇 분 늦는 상황이라면 재시도가 파일 도착을 기다리는 장치가 되지만, 파일이 아예 오지 않는 날이라면 재시도는 실패 통보를 늦추기만 한다. 어느 쪽인지는 원천의 지연 패턴이 정한다.

## 산출물

일별 실행은 `daily/{날짜}/` 아래에 보고서(`report.md`), 요약(`daily_summary.json`), 품질 기록(`quality_check.json`), 제외 상세(`excluded.json`), 완결 마커(`_SUCCESS`)를 남긴다. 주간 실행은 `weekly/{창 끝 날짜}/` 에 롤업 보고서와 요약을 남기고, 전 시나리오의 태스크 상태·해시·판정은 `ch7_batch_run_report.json` 하나에 모인다.

## 8. 핵심 요약

- 확정치와 근사치는 서로 다른 산출물이고, 마감을 정하는 주체가 다르다.** 스트리밍은 데이터가 워터마크로 마감을 정하므로 늦게 온 것을 버리고, 배치는 달력이 마감을 정하므로 그 시각까지 도착한 지연분을 전부 담는다. 두 수치가 달라도 왜 다른지가 문서로 설명되면 결함이 아니라 설계다.
- 정제한 레코드는 전부 행방을 적고, 합이 맞는지 식으로 검사한다.** `physical = mapped + missing + unmapped + dup_removed` 가 성립하지 않으면 어딘가에서 레코드가 조용히 사라진 것이다. 사흘치 실측  $60 = 54 + 2 + 3 + 1$ 이 이 검산의 결과이며, 중복을 뺀 `valid` 는 `physical - dup_removed` 다.
- 미기재와 미매핑은 합치지 않는다.** 미기재는 접수 화면이 지역을 받지 못한 것이고, 미매핑은 값은 왔는데 사전에 없는 것이다. 조치 대상이 접수 시스템과 사전으로 갈리므로, 둘을 "실패 2건"으로 합치면 판단에 필요한 정보가 사라진다. `판교 · 성남시 · 김포` 를 비슷한 자치구로 끼워 맞추는 것은 데이터 조작이다.

4. 품질 게이트는 틀린 산출물을 회수하지 않고 애초에 만들지 않는다. 총계 보존식이 깨지면 `check_quality` 가 예외를 던지고 `write_report` 는 `upstream_failed` 로 실행되지 않는다. 7월 4일 실행에서 하루 네 태스크가 모두 `upstream_failed` 로 기록된 것이 이 구조가 실제로 동작한 증거다.

5. 같은 입력의 재실행은 같은 바이트를 내야 하고, 상위 집계는 확정 산출물만 읽는다. 생성 시각을 넣지 않고, 키 순서를 고정하고, 덮어쓰기로 쓰면 해시가 재실행 전후에 같다. 주간 롤업은 `_SUCCESS` 마커가 있는 날짜만 합산해 일별 값과 주간 값이 어긋나지 않게 한다.

## 9. 과제

실습을 실행해 생성된 `practice/chapter7/data/output/ch7_batch_run_report.json` 을 LMS에 그대로 업로드하고, 다음 세 가지를 함께 제출한다.

1. 사흘 중 날짜 하나를 골라 그 날짜의 `quality` 블록에서 `physical`, `valid`, `mapped`, `missing`, `unmapped`, `dup_removed`, `gate`, `preservation_ok` 를 옮겨 적고, 총계 보존식의 각 항이 본인 값에서 무엇인지 밝힌 뒤 등식이 성립함을 산술로 보인다.
  2. `missing` 과 `unmapped` 를 하나로 합쳐 "실패 2건"으로 보고하면 안 되는 이유를, 두 항이 각각 무엇을 뜻하는지와 연결해 세 문장 이내로 쓴다.
  3. 본인 값이 표 7.1과 같은지 다른지 적고, 같다면 입력이 무엇이기에 결과가 결정적인지 설명한다.
- 수업일 포함 7일 이내에 제출한다.

## 10. 더 알아보기

이 장에서 다루지 않은 내용은 실무교재 `docs/chapter7.md` 에 있다.

- 배치 주기와 실행 시각을 무엇에서 정하는가 — 신선도 요구와 비용의 절충, 원천 확정 시각에서 역산하는 방법: `docs/chapter7.md` §7.1
- 데이터 품질 검증 프레임워크 — Great Expectations와 dbt 테스트로 총계 보존식을 선언적으로 관리하는 방법: §7.2
- 재시도 정책과 소급 실행 — 재시도 횟수·간격을 정하는 근거, `catchup` 과 `backfill`의 차이: §7.3
- 입력·출력 계약과 배치 실패 대응 체크리스트 — 발견부터 재발 방지까지 7단계: §7.4
- 태스크 분할·재시도·역등성을 스스로 설계하는 연습표(표 7.5)와 실행 중 겪은 DAG 직렬화 오류: `docs/chapter7.md` 실습 7.1, `practice/chapter7/`